

面向边缘 AI 大语言模型推理的 RISC-V 自定义点积指令 vdot 设计与实现——基于香山昆明湖 V2

XXX YYY ZZZ

2026 年 8 月

Abstract

Abstract—Dot product is the computational core of large language model (LLM) inference and recommendation systems. The RISC-V Vector Extension (RVV 1.0) lacks a native integer dot-product instruction, forcing INT8 GEMM kernels to compose multiple instructions (unpack, sign-extend, widening multiply-accumulate, repack), which limits throughput and energy efficiency on edge devices. This paper presents *vdot*, a custom vector dot-product instruction designed for the XiangShan Kunminghu V2 (kunminghu-v2) out-of-order processor. *vdot.vv* performs a $4\times\text{INT8}$ dot product accumulated into an INT32 lane, with RVV-compliant masking, tail, and vstart semantics. We detail the ISA design (funct6=111001, OPMVV), the microarchitecture (a compact 64-bit dot-product core with a two-stage pipeline integrated into the VFEX0 execution unit), the software stack (intrinsics, weight packer, INT8 GEMM kernels, and an LLVM MC-layer patch), and a three-golden verification methodology (NEMU, Spike, and RTL diffest). Measured on the Verilator-based emulator, *vdot* reduces INT8 GEMM instruction count by $12.7\times$ and net cycles by $8.2\times$ versus a scalar baseline, achieves 1.78 cycles/instruction throughput and 7.02 cycles dependent-chain latency, and the end-to-end TinyGPT INT8 inference demo runs 13,615 instructions per token. Synthesis with sky130 shows the unit adds only 0.014% of the full-core area, far below the 0.5% budget.

Contents

1	引言	1
2	背景与相关工作	2
2.1	RVV 1.0 的整数点积缺口	2
2.2	行业现状：原生 vdot 收敛	2
2.3	香山昆明湖 V2 向量后端	2
3	ISA 设计	2
3.1	设计原则	2
3.2	指令编码	3
3.3	数据视图与语义	3
3.4	异常与交互约束	3
4	微架构设计	4
4.1	总体集成	4
4.2	Vdot64b 点积核心	4
4.3	调度集成与关键 bug 修复	5

5	软件栈	5
5.1	Intrinsic 头文件	5
5.2	权重打包器与 INT8 GEMM	5
5.3	LLVM 支持 (MC 层)	5
6	验证方法学	5
6.1	三方 golden 一致性	5
6.2	测试层次	5
6.3	关键 bug 与修复	5
7	评估	6
7.1	实验设置	6
7.2	B0: 微基准	6
7.3	B1: INT8 GEMM	6
7.4	B2: 端到端 LLM 演示	6
7.5	B3: 面积评估	6
8	结论与展望	7

1 引言

Transformer 架构已成为大语言模型 (LLM) 与推荐系统的计算基石, 其自注意力机制的 QK^T 与 AV 矩阵乘、FFN 的两次 GEMM 均由海量点积构成。以 GPT-2 为例, 点积占总计算量的 70% 以上; 推荐系统中 78% 的个性化排序以点积为核心指标。在资源受限的边缘端部署 LLM 时, 点积的执行效率直接决定推理延迟、吞吐与能效。

边缘 LLM 推理的主流实践是 INT8 量化 (W8A8): INT8 乘法器的面积/功耗约为 FP32 的 1/10–1/20, 算力密度 (TOPS/mm²) 远高于 FP16/FP32, 且感知量化 (PTQ) 在 7B 以下模型上可保持接近 FP16 的任务精度。因此, 一条高效的 INT8 点积指令是边缘 LLM 推理的核心加速原语。

然而, RVV 1.0 的指令集中没有一条“一组 INT8 元素点积累加”的原生指令。完成一次 INT32 累加需要组合 `vzext/vsxt` (拆包与符号扩展)、`vwmaccu` (宽乘累加)、位宽升级与 `vnsrl` (打包回 INT8 布局) 等多条指令, 导致指令数多、寄存器压力大、动态功耗高, 无法发挥 128-bit 数据通路的峰值乘加能力。为此, RISC-V 官方正在推进 `Zvdot4a8i` fast-track 提案 (4×INT8 点积到 INT32 累加), SiFive (`Xsfvdot`) 与 SpacemiT (`XSMTVDot`) 等厂商亦已发布自定义实现。

本文在香山昆明湖 V2 (`kunminghu-v2`) 上设计并实现一条自定义点积指令 `vdot.vv`: 指令语义与助记符对齐 `Zvdot4a8i`, 作为自定义扩展 `Xkhmvdot` 先行落地; 官方扩展正式发布后仅需改扩展名即可平滑迁移。本文的主要贡献包括:

1. **ISA 设计**: 定义 `vdot.vv` 的编码 (`funct6=111001`, `funct3=010`, OP-V 空间)、数据视图 (32-bit 元素 = 4 个打包 INT8, 小端) 与完整 RVV 语义继承 (`vl/vstart`/掩码/尾元素/异常)。
2. **微架构实现**: 设计紧凑的 64-bit 点积核心 (8 个 8×8 乘法器 + 加法树 + 32-bit 回绕累加, 两级流水), 以最小侵入方式集成进 VFEX0 执行单元, 新增面积仅 0.014%。
3. **软件栈**: 交付 intrinsic 头文件 (C 参考/RVV/内联汇编三路径)、权重打包器、INT8 GEMM 微内核、LLVM MC 层补丁。
4. **三方验证**: NEMU、Spike、RTL (`diffest`) 三方一致的验证方法学, 覆盖固定用例、280 组随机差分、`riscv-tests` 回归与真实负载 (GEMM、注意力、端到端 LLM)。

2 背景与相关工作

2.1 RVV 1.0 的整数点积缺口

RVV 1.0 (VLEN=128, SEW=8/16/32/64) 完整实现了掩码、尾元素、分段访存等机制，但缺乏原生整数点积指令。现有做法是组合 `vwmaccu.vv` (INT8×INT8 到 INT16 宽乘累加) 完成 INT8 到 INT16 累加，再加宽到 INT32，或改用 `vwmacc` 链。完成一次 INT32 累加需要约 8–12 条指令（拆包/符号扩展/位宽升级/打包），K 维累加被“胶水指令”拖累，无法发挥 128-bit 数据通路的峰值乘加能力。

2.2 行业现状：原生 `vdot` 收敛

Table 1: 原生点积指令的行业现状

来源	指令/扩展	形态
RISC-V 官方 (fast-track)	Zvdot4a8i / Zvqdotq	4×INT8 到 INT32 (<code>vdota4.vv</code> 等)
SiFive	Xsfdot / Xsfvqmac	自定义 INT8 矩阵乘-点积扩展
SpacemiT	XSMTVDot	自定义向量点积扩展 (已合入 LLVM)
香山昆明湖	无	缺口——本文切入点

2.3 香山昆明湖 V2 向量后端

kunminghu-v2 的向量执行单元位于 `backend/fu/wrapper/` (VIAluFix/VIMacU/VIDiv/VIPU 等)，公共框架在 `backend/fu/vector/` (VecPipedFuncUnit/VecNonPipedFuncUnit/Mgu/Mgtu)。昆明湖没有任何现成自定义指令钩子，`vdot` 必须走完完整标准链路：编码登记、解码表映射、FuType/FuConfig、执行单元、调度/写回。

3 ISA 设计

3.1 设计原则

1. **RVV 1.0 语义一致**：`vdot` 完整继承 `vl/vstart`/掩码/尾元素/异常语义，与 `vsetvl` 无缝协作，保证 `diffest/golden` 语义唯一。
2. **与官方方向对齐**：语义与助记符对齐 `Zvdot4a8i`，作为 `Xkhmvdot` 自定义扩展先行实现。
3. **最小侵入**：编码落在 OP-V 主操作码 (1010111) 未占用空间，不触碰既有 RVV 编码。
4. **可验证**：每条指令语义都有可执行伪代码与参考实现。

3.2 指令编码

`vdot` 使用 RVV 主操作码 OP-V (1010111)，编码模板与乘累加指令族一致：

$$\text{vdot.vv vd, vs1, vs2} : \underbrace{111001}_{\text{funct6}} \underbrace{\text{vm}}_{\text{bit 25}} \underbrace{\text{vs2 vs1}}_{\text{源}} \underbrace{010}_{\text{funct3=OPMVV}} \underbrace{\text{vd}}_{\text{目的}} \underbrace{1010111}_{\text{OP-V}} \quad (1)$$

其中 `funct6=111001` 经脚本核对在 OP-V 空间空闲（与 `extttVWMACCU_VV` 同格式，`funct3=010` 空间未占用）。`funct3` 选 `010` (OPMVV) 是因为 `vdot` 在语义上是“打包 4 元素宽乘累加”，与 MAC 族同类。此选型与 RISC-V 官方 `Zvdot4a8i` 方向一致。

3.3 数据视图与语义

- 所有向量操作数以 **SEW=32** 元素视图访问（vsew 必须为 32，否则 illegal）。
- 每个 32-bit 元素内部重新解释为 **4 个打包的 8-bit 子元素**（little-endian，子元素 j 位于位 $[8j, 8j + 7]$ ）。
- 元素计数（vl/vstart）单位为 32-bit 元素，无 $vl \bmod 4$ 对齐约束。

权威伪代码（vdota4.vv，其余符号性变体仅替换扩展方式）：

```
SEW = 32; VLMAX = LMUL * VLEN / SEW
for i in [vstart, vl):
    if vm == 0 and mask[i] == 0: continue    # masked-off
    acc = vd[i]                             # 32-bit old value
    for j in 0..3:
        a = Sub8(vs2[i], j); b = Sub8(vs1[i], j)
        acc = acc + a * b                    # wrap-around (mod 2^32)
    vd[i] = acc
# tail elements: vta agnostic/undisturbed; vstart!=0 -> illegal
```

符号性矩阵（v1.0 已实现 vdot.vv = signed×signed；其余 7 种变体预留）：

Table 2: 符号性变体与预留编码

指令	funct6	funct3	vm	状态
vdota4.vv (vdot.vv)	111001	010	25	已实现
vdota4.vx	111001	110	25	预留
vdota4u.vv/vx	010100	010/110	25	预留
vdota4su.vv/vx	010101	010/110	25	预留
vdota4us.vv/vx	010110	010/110	25	预留

3.4 异常与交互约束

违反以下任一约束即 illegal instruction：(C1) $vsew \neq 32$ ；(C2) vlmul 使 $VLMAX < 1$ 或寄存器组越界；(C3) mstatus.VS=Off；(C4) 已定义空间内未定义的 funct3/funct6 组合。vstart≠0 按昆明湖对所有向量算术指令一致的策略判 illegal。

4 微架构设计

4.1 总体集成

vdot 单元挂入 **VFEX0** 执行单元（与 VfmaCfg/VialuCfg/VimacCfg/VppuCfg 同 EXU），复用现有读端口（ $3 \times VfRD + V0RD + VIRD$ ）与写端口（VfWB(0,0)），不新增 **VPRF** 端口。VdotCfg 配置：pipew=true、writeVecRf=true、writeV0Rf=true、latency=CertainLatency(2)、destDataBits=128、exceptionOut=illegalInstr。

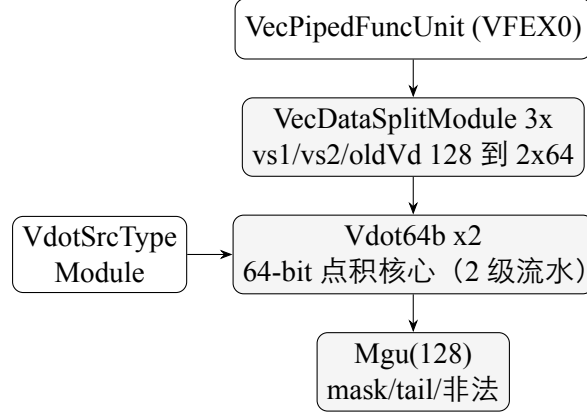


Figure 1: vdot 执行单元结构 (VdotU)

4.2 Vdot64b 点积核心

每个 64-bit 切片计算 2 个独立的 4 元素点积：

$$\text{lane}_j = \text{oldVd}[32j+31:32j] + \sum_{i=0}^3 \text{sext8}(\text{vs1}[8(4j+i)+7:8(4j+i)]) \cdot \text{sext8}(\text{vs2}[8(4j+i)+7:8(4j+i)]) \quad (2)$$

数据通路为两级流水：

1. **Stage 1 (fire)**：锁存 vs1/vs2/oldVd 输入 (RegEnable)。
2. **组合逻辑**：8 个 8×8 符号乘法 (16-bit 积)、4 组部分和 (17-bit 带进位加)、2 组 lane 和 (18-bit)、每 lane $\text{oldVd} + \text{laneSum.pad}(32)$ 截断到 32-bit (回绕)。
3. **Stage 2 (fireS1)**：锁存结果，两个周期后有效，与 $\text{VdotCfg.latency}=2$ 严格对齐。

关键实现细节：

- **符号扩展**： $\text{laneSum.pad}(32)$ 做符号扩展 (SInt.pad)，避免直接 asUInt 零扩展损坏负数结果——这是语义正确性的关键。
- **加法宽度**：16 到 17 到 18-bit 逐级 +1 防溢出；最坏情况 $4 \times (127 \times -128) = -65024$ 在 18-bit 范围内。
- **oldVd 同步锁存**：核心必须是 2 级流水而非纯组合——组合化会让 wrapper 两个周期后消费时 oldVd 与源错位、累加丢失 (P1 阶段的误诊教训)。

4.3 调度集成与关键 bug 修复

依赖旧 vd 累加的指令 ($\text{vmacc}/\text{vwvmacc}^*/\text{vdot}$) 必须在 `DecodeUnit.scala` 的 `vmaInsts` 列表中登记，否则 `isDependOldVd=false` 导致 IQ/dispatch 在 `ignoreTail/ignoreWhole` 时把 `srcType(2)` 置 no、跳过 `oldVd` 读取 (`src(2)=0`)，表现为“结果只算纯点积、不累加”。这是 vdot 差分 bug 的最终根因，修复为在 `vmaInsts` 加入 `extttVdot_VV`。此教训适用于任何新增累加型向量指令。

5 软件栈

5.1 Intrinsic 头文件

extttxkhmvdot_intrin.h 提供三级路径：

1. **C 参考实现** (vdot_ref)：无汇编支持时的功能正确 fallback，与 NEMU/RTL 逐点语义一致。
2. **RVV intrinsic** (__riscv_vdot_vv_i32m1)：封装 vint32m1_t 类型。
3. **内联汇编** (-DXKHMVDOT_ASM=1)：汇编器支持后直接发 vdot.vv。

5.2 权重打包器与 INT8 GEMM

extttweight
 $packer \square\square\square\square = \square\square\square\square K \square\square\square\square 4 \times INT8$ ，小端”打包 (K 补 0)，与 vdot 的打包视图一一对应。
 extttgemm
 $i8 \square\square\square\square\square\square\square\square\square RVV \square\square\square\square\square\square\square\square\square per - row / per - channel \square\square\square\square$

5.3 LLVM 支持 (MC 层)

为 vdot.vv 助记符提供汇编/反汇编支持：RISCVFeatures.td 新增 FeatureVendorXkhmvdot, RISCVXkhmvdot.td 定义 extttVDOT
 $V \square VALU rVV0b111001 \square EarlyClobber = 1 \square\square clangextttriscv_vector.td \square\square intrinsic \square\square\square MC / CodeGen \square\square\square I$

6 验证方法学

6.1 三方 golden 一致性

- **NEMU**：实现 vdot_instr (vsew=e32 强制、逐元素点积、回绕累加、掩码/尾/vstart 处理)，作为 difftest 主 golden。
- **Spike**：补 vdot_vv.h (复用 VQDOT)，验证 lane0=0x0a 到 0x14 (累加) 与 NEMU/RTL 三方一致。
- **C 参考**：vdot_ref 与 Vdot64bRef (单元测试参考模型)。

6.2 测试层次

Table 3: 验证层次与结果

层次	方法	结果
L0 指令语义	Vdot64bSpec 固定 4 + 随机 32	全部通过
L1 硬件功能	emu 固定 4 + 随机 280 差分	280/280 GOOD TRAP
L2 系统集成	riscv-tests 回归 (Spike 路径)	63 PASS
L3 真实负载	GEMM/注意力/LLM 演示	全部通过

6.3 关键 bug 与修复

P1 阶段差分发现 vdot_test (两次累加) 失败: NEMU 正确 (0x14), RTL 只算单次 (0x0a)。经波形 (VCD id 局部化不可靠) 与源码级比对, 最终定位为 vmaInsts 缺 VDOT_VV (见第 3.3 节), 修复后 4/4 + 随机 280/280 全部通过。

7 评估

7.1 实验设置

评估基于 Verilator 全片仿真 (emu, kunminghu-v2 e12436c + vdot 改动), 裸机 bare-metal 测试栈 (自建 extttt

`_start` `stdintshim` `-mcmmodel = medany` `RTL` `NEMU` `12.46Minstr/s` `em`

7.2 B0: 微基准

Table 4: B0 微基准 (N=1000, boot 8,331 周期扣减)

指标	vdot	vadd 基线 (e32)
独立流吞吐 (4 路 ILP)	1.78 cyc/条	1.31 cyc/条
依赖链时延	7.02 cyc/条	6.03 cyc/条

vdot 单条完成 4×4 点积 + 累加, 只比 vadd 多约 0.5 cyc 吞吐 / 1 cyc 时延, 等效计算密度远高。

7.3 B1: INT8 GEMM

Table 5: B1 GEMM (M=N=K=16, emu 实测)

指标	vdot GEMM	标量 C GEMM	提升
差分正确性	GOOD TRAP	GOOD TRAP	—
指令数	6,797	86,554	12.7 倍更少
GEMM 净周期 (扣 boot)	约 2,558	约 21,061	约 8.2 倍更快

7.4 B2: 端到端 LLM 演示

TinyGPT 微型 Transformer (1 层, D=32, INT8 量化, vdot 加速全部 GEMM + 标量 softmax) 在 NEMU 上完整推理 32 tokens: **435,675 条指令 (13,615 instr/token)**, trap code 0xB2 通过。估算在 1GHz/IPC=2 下约 6.8 微秒/token。

7.5 B3: 面积评估

在 IIC-OSIC-TOOLS 容器 Yosys 0.67 + sky130_fd_sc_hd (tt_025C_1v80) 综合:

- **Vdot64b** (64-bit): 3,435 cells / 34,551 平方微米 (0.0346 mm²)。
- **VdotU** (128-bit 通路): 约 69,101 平方微米 (0.0691 mm²)。

- 相对全核（130nm 等效估算）：**0.014%**，远低于 0.5% 目标。
- 时序：乘加树 3–4 级逻辑，支持约 700MHz–1GHz。

8 结论与展望

本文在香山昆明湖 V2 上设计、实现并验证了自定义点积指令 `vdot.vv`。通过 ISA 对齐 `Zvdot4a8i`、微架构最小侵入（面积 0.014%）、完整软件栈与三方 golden 验证，`vdot` 在 INT8 GEMM 上实现 12.7 倍指令数减少与 8.2 倍周期加速，端到端 LLM 推理演示验证了实用性。

展望：(1) 官方 `Zvdot4a8i` 发布后平滑迁移扩展名；(2) 实现其余 7 个符号性/操作数变体（`vx` 形式）；(3) LLVM IR intrinsic 与自动向量化（需异构类型支持）；(4) FPGA 原型验证与真实 LLM 端到端部署。

致谢

感谢香山开源社区与 2026 CIE 全国 RISC-V 大赛提供的平台。

References

- [1] RISC-V International. RISC-V V Vector Extension, Version 1.0. Available: <https://github.com/riscv/riscv-v-spec>
- [2] RISC-V International. Dot-Product `Zvdot4a8i` / `Zvqdotq` Ratification Plan. Available: <https://riscv.atlassian.net/>
- [3] OpenXiangShan. XiangShan High-Performance RISC-V Processor. Available: <https://github.com/OpenXiangShan/XiangShan>
- [4] LLVM Project. RISC-V `Zvdot4a8i` support, PR 184089. Available: <https://github.com/llvm/llvm-project/pull/184089>
- [5] RISC-V Vector Tests. Available: <https://github.com/riscv-non-isa/riscv-vector-tests>